LECTURE: 2 BASIC OF C++ (PART - 1)

Mondira Chakraborty

Lecturer, CSE Dept.

Notre Dame University Bangladesh

Content

1. What is C++?
2. Why C++?
3. Hello World!
4. Data types
5. C++ Arithmetic Operators
6. C++ Assignment Operators
7. C++ Comparison Operators
8. C++ Logical Operators
9. Conditions and Loops

What is C++?

- We all know that, C++ is a programming language
- C++ was developed by Bjarne Stroustrup, as an extension to the C language.
- It is a cross-platform language that can be used to create high-performance applications
- C++ gives programmers a high level of control over system resources and memory
- The language was updated 4 major times in 2011, 2014, 2017, and 2020 to C++11, C++14, C++17, C++20

Why C++?

- Allow us to express common sequences of commands more concisely.
- Modifying code is easier when it entails just a few text-edits, instead of rearranging hundreds of processor instructions.
- One of C++'s strengths is that it can be used to write programs for nearly any processor.
- It is a fast language; its compile-time and execution time is faster than other programming languages
- To solve advanced puzzles that require top performance, speed can be an advantage.

Hello World!

```
// A Hello World program
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, world!\n";
    return 0;
}
```

Hello World!

- **Line 1:** #include <iostream> is a **header file library** that lets us work with input and output objects, such as cout (used in line 5). Header files add functionality to C++ programs.
- **Line 2:** using namespace std means that we can use names for objects and variables from the standard library.
- **Line 3:** blank line. C++ ignores white space. But we use it to make the code more readable
- **Line 4:** Another thing that always appear in a C++ program, is int main(). This is called a **function**. Any code inside its curly brackets {} will be executed.

Hello World!

- **Line 5:** cout (pronounced "see-out") is an **object** used together with the *insertion operator* (<<) to output/print text. In our example it will output "Hello World!".
- **Line 6:** return 0 ends the main function.
- **Line 7:** Do not forget to add the closing curly bracket } to actually end the main function.

Hello World!

- **Note 1:** Everything in C++ is case sensitive: someName is not the same as SomeName.

- **Note 2:** // Indicates that everything following it until the end of the line is a comment: it is ignored by the compiler. Another way to write a comment is to put it between /* and */ (e.g. x = 1 + /*sneaky comment here*/ 1;). A comment of this form may span multiple lines. Comments exist to explain non-obvious things going on in the code. Use them: document your code well!

Hello World!

- **Note 3:** The body of int main() could also been written as:
  int main () { cout << "Hello World! "; return 0; }

- **Note 4:** The compiler ignores white spaces. However, multiple lines makes the code more readable.

- **Note 5:** Every C++ command ends with a semicolon ;

- **Note 6:** It is your decision if you want to add library function or not.

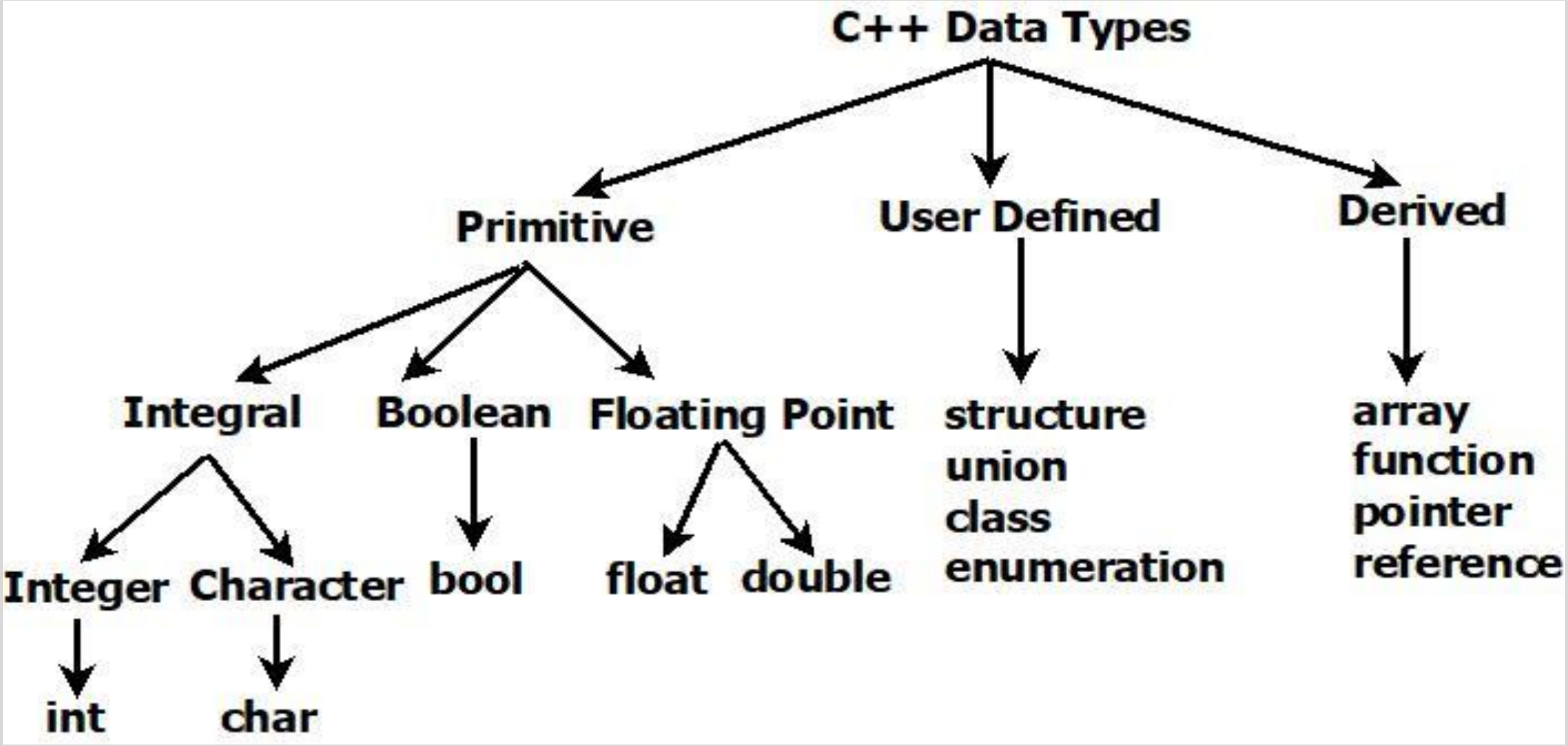
Input Output

```
1. #include <iostream>
2. using namespace std;
3.
4. int main() {
5.     cout << "Hello World!";
6.     cout << "I am learning C++";
7. return 0;
8. }
```

Input Output

```
1. int x, y;
2. int sum;
3. cout << "Type a number: ";
4. cin >> x;
5. cout << "Type another number: ";
6. cin >> y;
7. sum = x + y;
8. cout << "Sum is: " << sum;
```

Data types


C++ Data Types
Primitive
User Defined
Derived
Integral
Boolean
Floating Point
structure
union
class
enumeration
array
function
pointer
reference
Integer
Character
bool
float
double
int
char

Data types

Data Type	Size	Description
`boolean`	1 byte	Stores true or false values
`char`	1 byte	Stores a single character/letter/number, or ASCII values
`int`	2 or 4 bytes	Stores whole numbers, without decimals
`float`	4 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits
`double`	8 bytes	Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits

Data Types

Type	Definition	Control Character	Limits
int	Integer		-2147483648 to 2147483647
short	Short Integer		-32768 to 32767
long	Long Integer	l or L	-2147483648 to 2147483647
float	Floating Decimal Number	f or F	1.17549e-038 to 3.40282e+038
double	Double Decimal Number		2.22507e-308 to 1.79769e+308
long double	Long Decimal Number		2.22507e-308 to 1.79769e+308
char	Character		-128 to 127
unsigned int	Unsigned Integer		0 to 4294967295
unsigned short	Unsigned Short Integer		0 to 65535
unsigned long	Unsigned Long Integer		0 to 4294967295
unsigned char	Unsigned Character		0 to 255
bool	True or False		True = 1 and False = 0

Data types

Key word	Size in bytes	Interpretation	Possible values
bool	1	boolean	true and false
unsigned char	1	Unsigned character	0 to 255
char (or signed char)	1	Signed character	-128 to 127
wchar_t	2	Wide character (in windows, same as unsigned short)	0 to 2^{16}-1
short (or signed short)	2	Signed integer	-2^{15} to 2^{15} - 1
unsigned short	2	Unsigned short integer	0 to 2^{16}-1
int (or signed int)	4	Signed integer	- 2^{31} to 2^{31} -1
unsigned int	4	Unsigned integer	0 to 2^{32} - 1
Long (or long int or signed long)	4	signed long integer	- 2^{31} to 2^{31} -1
unsigned long	4	unsigned long integer	0 to 2^{32} -1
float	4	Signed single precision floating point (23 bits of significand, 8 bits of exponent, and 1 sign bit.)	$3.4*10^{-38}$ to $3.4*10^{38}$ (both positive and negative)
long long	8	Signed long long integer	-2^{63} to 2^{63} -1
unsigned long long	8	Unsigned long long integer	0 to 2^{64} -1
double	8	Signed double precision floating point(52 bits of significand, 11 bits of exponent, and 1 sign bit.)	$1.7*10^{-308}$ to $1.7*10^{308}$ (both positive and negative)
long double	8	Signed double precision floating point(52 bits of significand, 11 bits of exponent, and 1 sign bit.)	$1.7*10^{-308}$ to $1.7*10^{308}$ (both positive and negative)

C++ Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

C++ Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

C++ Comparison Operators

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

C++ Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Conditions and Loops Structures

Note 7: It is same as C, with some more flexibility

```
1. if (condition) {
2.     // block of code to be executed if the condition is true
3. }
/////////////////////////////
1. if (condition) {
2.     // block of code to be executed if the condition is true
3. else {
4.     // block of code to be executed if the condition is false
5. }
```

Conditions and Loops Structures

```
1. if (condition1) {
2. // block of code to be executed if condition1 is true
3. }
4. else if (condition2) {
5. // block of code to be executed if the condition1 is false
   and condition2 is true
6. }
7. else
8. {
9. // block of code to be executed if the condition1 is false
   and condition2 is false
10.}
```

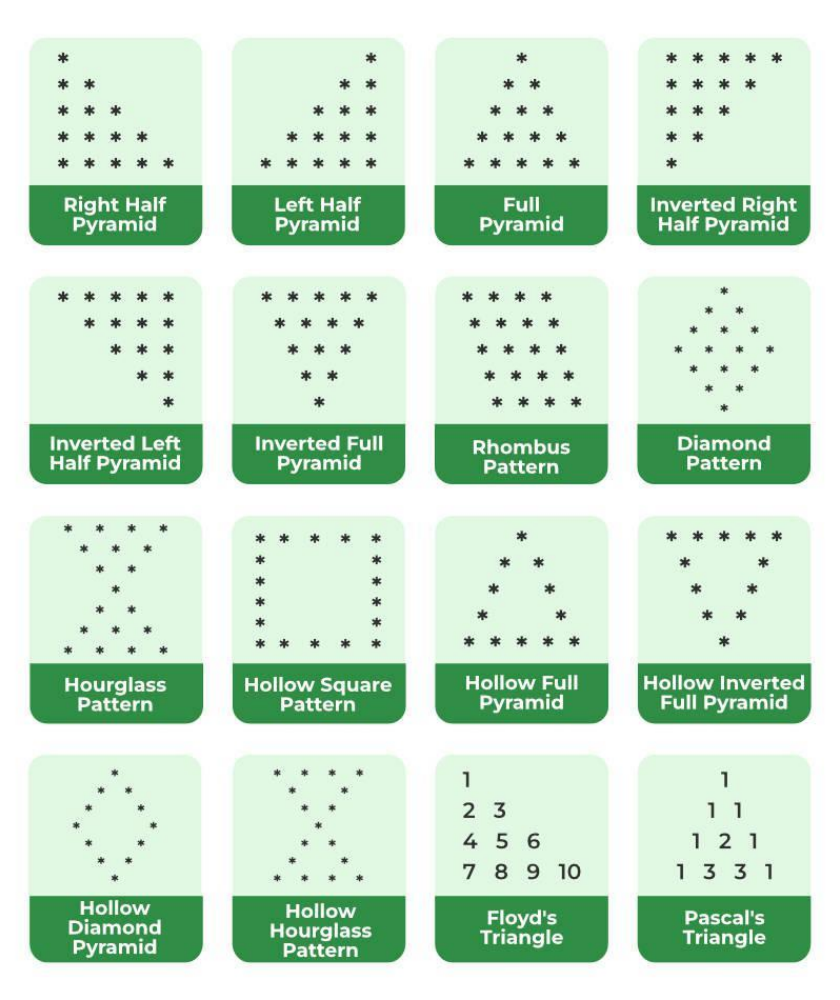
Conditions and Loops Structures

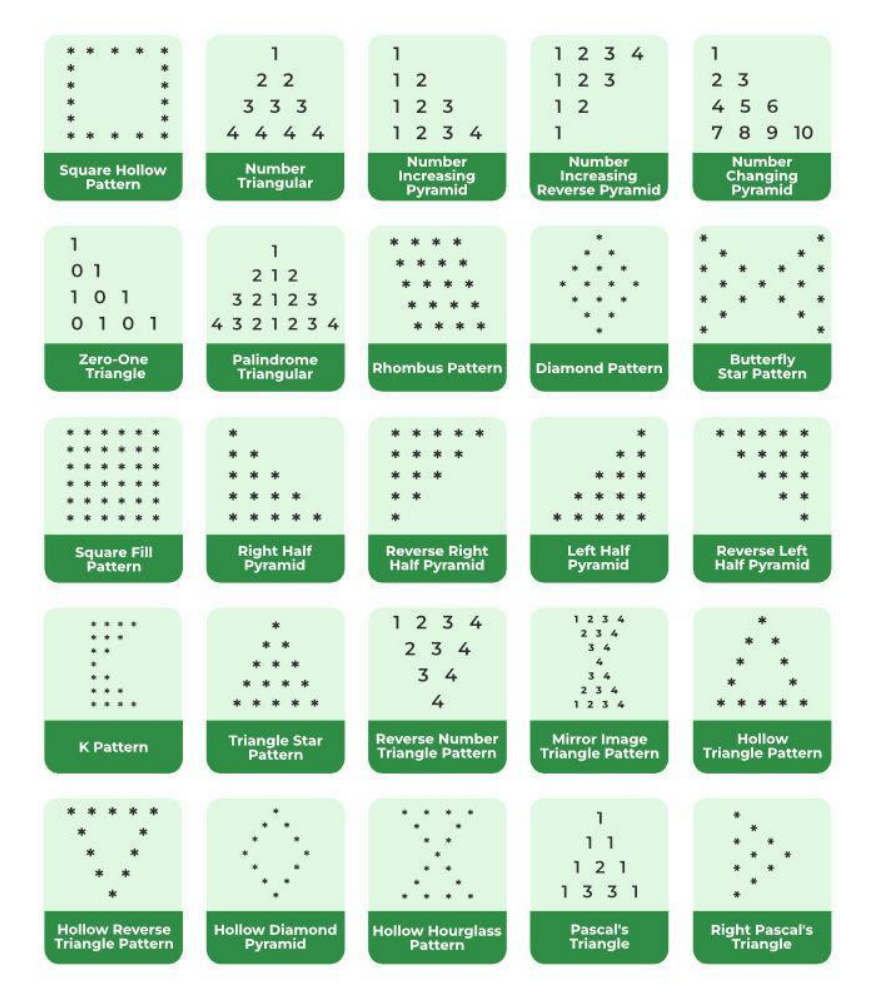
```
1. variable =(condition)? expressionTrue : expressionFalse;


////////////////////////////////////
1. int result;
2. cin >> result;
3. string r = (result < 70) ?  “Loser!”: “Barely Passing!”;
4. cout << r;
```

Conditions and Loops Structures

```
1. int i = 0;
2. while (i < n) {
3.     // constant time code here
4.     i++;
5. }
////////////////////////////////////
1. for (int i = 1; i <= n; i++) {
2.   // constant time code here
3. }
```

Try to find Logic!





References:

1. https://ocw.mit.edu/courses/6-096-introduction-to-c-january-iap-2011/resources/mit6_096iap11_lec01/
2. https://www.w3schools.com/cpp/default.asp
3. https://www.toptal.com/c-plus-plus/c-plus-plus-competitive-programming#:~:text=C%2B%2B%20is%20known%20for%20its,provides%20an%20extensive%20standard%20library.
4. https://www.geeksforgeeks.org/pattern-programs-in-c/